4-2 Milestone Three: Enhancement Two: Algorithms and Data Structures

Andrew Allen

Department of Computer Science, Southern New Hampshire University

CS 499: Computer Science Capstone

Dr. Sherin Aly

July 26th, 2026

Enhancement Two: Algorithms and Data Structures

Chosen Artifact

The artifact I chose for inclusion in my ePortfolio for this enhancement is an Android Inventory Tracking application I originally created during the CS360 Mobile Architect and Programming course. The application allows users to create, update, delete, and manage inventory items while tracking quantities and other item details. For this enhancement, I expanded the applications' algorithmic functionality by implementing more efficient collection processing, dynamic sorting, and real time calculations using Kotlin collections and modern android components.

Justification for Inclusion

I selected this artifact because it demonstrates how algorithms and data structures directly impact app performance and usability. The original app did successfully manage inventory records, but did not incorporate any searching, filtering, or data aggregation algorithms. Therefore, with this enhancement I emphasized these and incorporated them into the app to bring more value to the end user.

One of the primary enhancements was implementing dynamic multifield sorting using Kotlin collection operations. Rather than creating separate sorting routines for each field, I implemented comparator-based sorting that allowed users to organize inventory items by name, price, or quantity in either ascending or descending order. This approach performs efficient in-memory sorting with a time complexity of $O(n \log n)$ while reducing duplicated code and improving maintainability.

I also implemented real time aggregation algorithms within the ViewModel to continuously calculate inventory statistics. Each time the inventory data changes, the application performs a single linear pass through the collection to calculate the total inventory value and determine the number of items that have reached or fallen below their low-stock threshold. These operations execute in $O(n)$ time and automatically update the user interface through LiveData without blocking the main thread.

Another enhancement made to the application was implementing collection transformation pipelines for filtering and resetting inventory views. Instead of modifying the inventory collection directly, the app applies filter predicates and sorting operations to produce the transformed inventory collection to the user. Maintaining an immutable dataset improves data consistency and makes the apps' behavior more predictable and easier to maintain.

Course Outcome Coverage

The enhancements completed for this artifact successfully meet the algorithms and data structure outcomes I established originally for the artifact, in particular, course outcomes three and four.

For outcome three: Design and evaluate computing solutions using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices. My enhancements demonstrate this outcome by implementing efficient collection processing algorithms, comparator based sorting, and linear aggregation techniques to organize and analyze inventory data. Evaluating the time complexity of these operations allowed me to select approaches that provide the best performance while maintaining clean code.

For outcome four: Demonstrate an ability to use a well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals. The app was enhanced using Kotlin's collection framework, LiveData, ViewModel, and other modern Android development practices. These technologies allowed me to separate algorithmic processing from the user interface while creating a more scalable and maintainable application.

Reflection

Enhancing this artifact strengthened my understanding of how algorithms and data structures influence both the apps performance and user experience. While the original application stored and displayed inventory data correctly, this enhancement required me to think more about how collections could be manipulated efficiently while keeping the user interface responsive.

One of the most valuable lessons I learned during this enhancement was how Kotlin's higher order functions simplify complex algorithmic operations without sacrificing performance. Implementing comparator-based sorting, filtering, and aggregation algorithms reinforced the importance of selecting the appropriate algorithm rather than relying on repetitive or inefficient coding practices to achieve the desired results.

One of my biggest challenges during this enhancement was integrating these algorithms into a reactive app architecture. Since the inventory data automatically updates the user interface through LiveData and ViewModel components, I needed to ensure that sorting, filtering, and aggregation operations remained in sync with changes to the inventory data. Solving these

challenges improved my understanding of reactive programming, state management, and lifecycle aware app design and how to properly approach these architectures.